

VS-HGNN: Cross-Modal Heterogeneous Graph Learning for Multi-Label Smart-Contract Vulnerability Detection

End-to-end research architecture: data fusion (Step 1), heterogeneous graph construction, model, training and calibration (Step 2), inductive auditing (Step 3)

A Data sources

SmartBugs Wild (on-chain corpus)

972,975 Ethereum contracts, 194.7 M transactions
per contract: address, tx count, balance, creation
and last-transaction dates, compiler version, name

DIVE (verified multi-label ground truth)

22,330 labelled contracts, 8 DASP classes
 $y \in \{0,1\}^8$ per contract
6 static analysers $\times$ 8 classes = 48 tool flags
(MAIAN, Mythril, Semgrep, Slither, Solhint, VeriSmart)

B Step 1: cross-modal fusion

Address canonicalisation $\pi(a)$

$\pi(a) = \text{lowercase}(\text{strip}(a \setminus "0x")) \rightarrow \Sigma^{40}$
idempotent projection onto a common key space

Hash equi-join on $\pi(a) \Rightarrow$ supervised core

$D_{\text{core}} = D_{\text{DIVE}} \bowtie D_{\text{SB}}$ (inner join, de-duplicated)
7,487 contracts $\times$ 66 columns (33.5 % of DIVE)
runtime metadata + tool flags + ground-truth labels
134 distinct compiler versions

scripts/merge_datasets.py

$G = (x_{\text{dict}}, \text{edge_index_dict}, \text{train / val / test masks})$

C Heterogeneous graph G

scripts/construct_graph.py

Contract nodes (7,487)

$x \in \mathbb{R}^{72} = [\text{runtime (3)} \parallel$
tool priors (48) $\parallel$ compiler (21)]
compiler = one-hot top-20 + other
runtime = $\log_{10}$ tx, $\log_{10}$ balance,
lifecycle days (standardised)
labels $y \in \{0,1\}^8$ (from DIVE)

Compiler nodes (134)

identity one-hot, $x \in \mathbb{R}^{134}$

Relations (edge types)

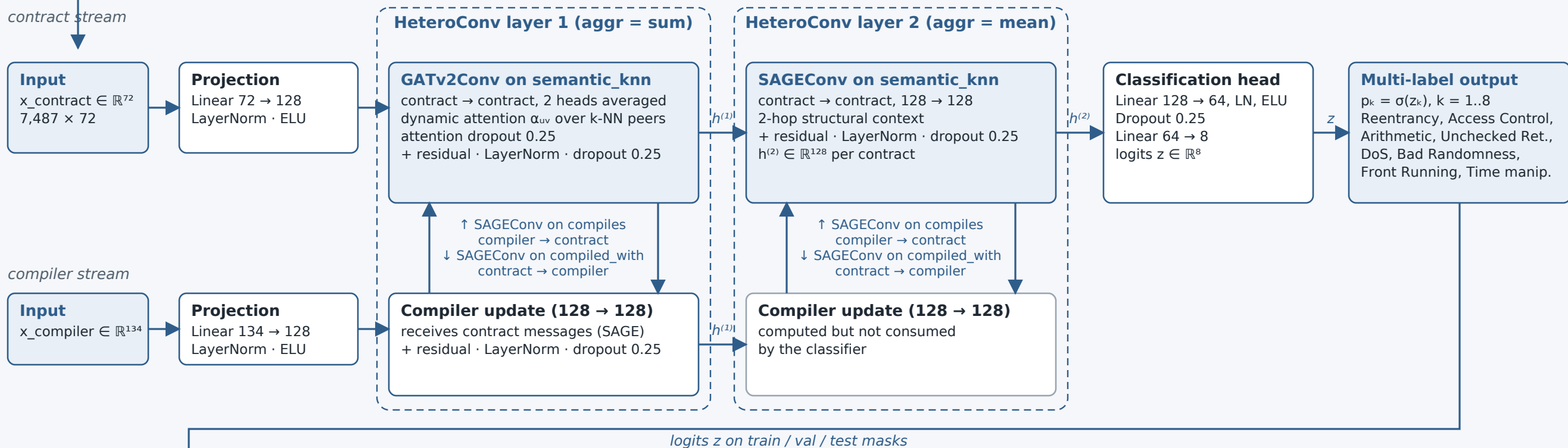
compiled_with / compiles
contract $\leftrightarrow$ compiler, 7,487 each
semantic_knn
contract $\leftrightarrow$ contract, cosine, $k = 5$
37,435 directed edges
7,621 nodes, 52,409 edges in total

Node split (seed 42)

train 5,240 (70 %) $\cdot$ val 1,123 (15 %)
test 1,124 (15 %), hold-out

D VS-HGNN model (class VSHGNN, PyTorch Geometric)

two node-type streams; three relation-specific message-passing operators per layer; residual + LayerNorm



E Step 2: training, evaluation, calibration, benchmarking

Weighted BCE loss (train nodes)

$w_k^+ = (N - N_k^+) / N_k^+$, clipped to [0.5, 50]
= [0.89, 0.50, 1.33, 2.71, 4.59, 33.7, 38.1, 2.52]
AdamW lr 5×10^{-3} , wd 10^{-4} , cosine LR schedule
grad-clip 2.0, ≤ 120 epochs, early stopping (20)
model selection on val macro-F1 (best ep. 106)

Threshold calibration (val, N = 1,123)

$\tau_k^* = \text{argmax over } \tau \in [0.05, 0.95] \text{ of}$
 $F_1(y_{\text{val},k}, 1[\sigma(z_{\text{val},k}) \geq \tau])$
 $\tau^* = [.53, .37, .54, .41, .81, .38, .45, .60]$
replaces the generic $\tau = 0.5$ decision rule
calibrated head micro-F1 0.9700 (vs 0.9647)

Hold-out test (N = 1,124)

Macro-F1 0.9633 Micro-F1 0.9647
mean ROC-AUC 0.9928 Hamming 0.0216
baselines on the same test contracts (macro-F1):
Slither 0.636 $\cdot$ Mythril 0.427 $\cdot$ Solhint 0.362
GCN on homogeneous k-NN graph 0.858
logistic regression 0.968 $\cdot$ random forest 0.971
significance vs static tools:
Wilcoxon $p = 0.0039$, McNemar $p < 10^{-15}$

Persisted artifacts (data/)

merged/vs_hgnn_supervised_benchmark.csv
graph/vs_hgnn_hetero_graph.pt
models/vshgnn_best.pt (frozen weights Θ^*)
models/eval_results.json
models/comprehensive_benchmark.json (τ^*)
public repository:
github.com/lahamisuahmed/VS-HGNN

scripts/train_vshgnn.py $\cdot$ scripts/calibrate_and_benchmark_baselines.py

F Step 3: inductive scanner and GNN explainer

Unseen contract u^*

runtime profile, static-tool
flags, compiler version
 $\rightarrow$ same feature map and
scaler as in training:
 $x_{u^*} \in \mathbb{R}^{72}$

Inductive insertion

cosine k-NN, $K = 5$ peers $\rightarrow$
bidirectional k-NN edges
+ compiled_with / compiles
edge to compiler node c_{u^*}
 $O(1)$ insertion, no retraining

Frozen VS-HGNN Θ^*

forward pass on the
augmented graph
 $\rightarrow z_{u^*} \in \mathbb{R}^8$
sub-40 ms latency

Calibrated decision

$\hat{y}_k = 1[\sigma(z_k) \geq \tau_k^*]$
margin $\Delta_k = \sigma(z_k) - \tau_k^*$
risk level from flagged
classes and margins

Explainer

Input $\times$ Gradient saliency
 $s_{k,i} = |\partial z_k / \partial x_i| \cdot |x_i|$
aggregated by modality:
runtime / tools / compiler
GATv2 attention $\alpha \rightarrow$ peer
influence (labelled peers)

Audit report

flagged DASP classes
confidence margins
modality attribution
top influencing peers
remediation notes

scripts/scan_contract.py

$\rightarrow$ learned representations / model flow

$\rightarrow$ supervision: ground-truth labels, loss, calibrated thresholds

$\dashrightarrow$ reuse of persisted artifacts

All figures from repository artifacts (seed 42)